

# Relatório

## Atividade 1: Servidor de Consultas Financeiras (Unicast)

O código implementa um cliente TCP simples que tem como função estabelecer uma conexão com um servidor remoto, enviar uma solicitação inserida pelo usuário e exibir a resposta recebida. E um servidor TCP monothread que atua como uma API de consulta financeira simplificada. Ele armazena dados de Fundos Imobiliários (FIIs) em memória e responde a comandos específicos enviados pelo cliente.

Primeiramente o IP de destino e a porta são definidos para o cliente e o servidor e o servidor fica em modo de escuta (listen), aguardando conexões.

O código utiliza a família de endereços AF\_INET (IPv4) e o tipo SOCK\_STREAM (protocolo TCP), garantindo a entrega confiável dos dados. Na parte cliente é solicitado que o usuário digite uma mensagem via console (input). No servidor, ao aceitar uma conexão, entra em um loop while True para receber dados continuamente, recebe a mensagem em bytes e a decodifica para string.

O cliente conecta-se ao servidor especificado, codifica a mensagem do usuário para o formato de bytes (UTF-8) e a envia. O servidor espera uma string formatada com um separador de ponto e vírgula (ex: COMANDO;TICKER). Utiliza .split(";") para separar o comando do código do ticker. A informação processada é enviada de volta ao cliente.

O servidor utiliza um dicionário Python para armazenar as informações, onde a chave é o *ticker* do ativo (ex: 'HGLG11') e o valor é outro dicionário contendo 'Preço' e 'Provento'.

## Atividade 2: Servidor de Leilão (Multithread+Broadcast)

O código implementa um sistema de leilão distribuído via TCP. A parte cliente permite que o usuário envie lances numéricos e receba atualizações em tempo real sobre o status do leilão através de uma thread dedicada de escuta. A parte servidora é multithread, capaz de gerenciar múltiplas conexões simultâneas, validando se os lances recebidos superam o valor atual e notificando os participantes (Broadcast) quando um novo recorde é atingido.

Primeiramente o IP de destino e a porta são definidos para o cliente e o servidor e o servidor fica em modo de escuta (listen), aguardando com uma fila de até 10 conexões.

O código utiliza a família de endereços AF\_INET (IPv4) e o tipo SOCK\_STREAM (protocolo TCP), garantindo a entrega confiável dos lances e mensagens de controle.

O cliente opera com duas threads: a principal captura os lances digitados pelo usuário (input), enquanto uma thread secundária (ouvir\_servidor) fica aguardando mensagens do servidor para exibí-las imediatamente. No servidor, cada nova conexão aceita dispara uma nova thread (worker), permitindo que múltiplos usuários enviem lances ao mesmo tempo.

O cliente envia o valor do lance codificado em bytes (UTF-8). O servidor decodifica, converte para número real (float) e processa a lógica de negócios. Se o lance for o novo maior, uma resposta de sucesso é enviada ao remetente e uma mensagem de aviso é disparada para todos os outros clientes conectados (realizar\_broadcast). Se for menor, apenas o remetente recebe a recusa.

O servidor mantém o estado do leilão em variáveis globais (MAIOR\_LANCE e lista CLIENTES\_CONECTADOS). O código utiliza um objeto mutex (Lock) para garantir a exclusão mútua (thread-safety), impedindo que dois clientes alterem o valor do lance ou a lista de conexões simultaneamente, evitando conflitos de dados.